VERBORUM

Push notifications Implementation guideline

Silent background sync for Android, web and iOS. Revision 2, with the owner decisions.

Document	Research, decisions and implementation guideline
Date	2026-10-14 (revision 2; revision 1 was 2026-10-13)
Scope	verborum_ms, verborum_android, verborum_kmp
Writing standard	ASD-STE100 style: short sentences, active voice, one idea per sentence
Status	Decisions recorded. No code is built. Tasks P4N-01 to P4N-13 are in the roadmap (Phase 4N).
Companion files	docs/agent/push-notifications.md (the normative guideline), docs/agent/roadmap.md

1. Decisions

Topic	Decision (owner, 2026-10-14)
Provider	**FCM direct** (Firebase Cloud Messaging, HTTP v1, Firebase Admin SDK for Java).
Where the code lives	A **new service, ms_notification**. Port 8088. Database vdbnotification on host port 5435.
Message kind	**Silent data messages only.** The app shows no alert. The goal is to keep the app up to date in the background.
First category	**SYNC**: a change on one device nudges the other devices of the same user.
Second category	**REFRESH**: a silent nudge to the publisher on import received and rating received.
Cancelled	Push to the importers of a dictionary. The event dictionary.unavailable. Importers are read-only and the apps sync on open.
Android	First client. No permission prompt and no notification channel are needed.
Web	**No FCM.** Refetch when the tab becomes visible, and on a slow timer while visible. SSE is a backlog item (BL-05).
iOS	Firebase SDK. Built last. Needs an Apple Developer account, which the owner does not have yet.
Order of work	Backend and Android (local) > web refetch > Android and web release > iOS build and release.

Answers to the three first questions

Question	Answer
Can I test locally?	**Yes.** The backend, the Android emulator and the iOS simulator run on your Mac. Only FCM is external, and it is free. Firebase has **no local emulator** for messaging.
How much does it cost?	**0 USD per message** with FCM. iOS needs the Apple Developer Program, **99 USD per year**. You need the same membership to publish on the App Store.
Which options exist?	Five. See section 4. FCM direct is the choice.

> **Core rule.** A push message wakes the app. It is not the data. The payload holds ids only. The app fetches the data from the API with its own token. A push is a hint: if a push is lost, the next sync repairs the data.

2. What exists today

Part	State (verified in the repo)	Effect on push work
Backend verborum_ms	ms_dictionary, ms_user, ms_marketplace. RabbitMQ topic exchange `verborum.events`. No device-token store. No push sender. Keycloak JWT on every endpoint.	Add ms_notification with a token store and a sender. Reuse the event bus.
Android verborum_android	Kotlin 2.1.0, AGP 8.7.0, Hilt, Room, WorkManager 2.9.1. Package `de.coldtea.verborum.app`. No Firebase. A `PostLoginHook` multibinding exists in core.	Add Firebase Messaging. Register the token in a PostLoginHook. Enqueue the existing sync worker.
KMP verborum_kmp	Kotlin 2.4.10, Compose Multiplatform 1.11.1. Targets: iOS and web. **No Android target.** Web keeps no local database. No entitlements file in iosApp.	Web: refetch on focus, no Firebase. iOS: capability and Swift bridge, later.

Events

Event	State	Used for
`dictionary.changed`	**New** (ms_dictionary). Any change to a user's dictionary or words, after commit, on change only.	SYNC to the other devices of that user.
`dictionary.imported`	Exists. Add `publisherId` (the payload holds the importer only).	REFRESH to the publisher.
`dictionary.rated`	**New** (ms_marketplace). On a new or changed rating only.	REFRESH to the publisher.
`dictionary.updated`	Exists. Covers only public listing fields.	Not used. It cannot serve sync.
`user.deleted`	Exists.	Delete all tokens of the user (match on keycloakId).

3. How silent push works

Step	What happens
1	The app signs in. The Firebase SDK gives the app a **registration token**.
2	The app sends the token to ms_notification, with a device id and the access token. The service stores it for that user.
3	A user changes a dictionary on device A. ms_dictionary commits and publishes `dictionary.changed`.
4	ms_notification finds the other devices of the user. It skips device A.
5	It sends a **data-only** message with FCM. FCM delivers it through Google (Android) or APNs (iOS).
6	The app receives the message in the background. It enqueues a sync. It shows nothing.

- Data messages show no alert. Android needs no `POST_NOTIFICATIONS` permission for them.
- Use **normal priority** and the collapse key `sync`. Ten quick edits then leave one pending message.
- Maximum payload is **4096 bytes**. Send ids only.
- Android tokens expire after 270 days without use. FCM treats a token as stale after about 30 days. iOS tokens do not expire, so you must clean them.
- Delete a token when FCM returns `UNREGISTERED` (HTTP 404).

> **Limit.** The OS can drop a silent push. Force-quit, Doze, Low Power Mode and Background App Refresh off all stop delivery. iOS limits silent pushes the most. The app must never depend on a silent push. It keeps its full sync on open.

4. Options considered

Provider

Option	Cost	Result
A. FCM direct	0 USD per message. Quota 600,000 messages per minute.	**Chosen.** One sender for all platforms.
B. FCM plus direct APNs	0 USD plus Apple membership.	Two senders. Not needed.
C. OneSignal	Free for 1,000 monthly active users, then from 19 USD per month.	Rejected. Cost grows. User ids go to a third party.
D. Self-hosted Web Push	Hosting only.	Rejected. Web only.
E. No push (polling, live connection)	Server load and battery.	Used for web only (see below).

Web freshness

#	Option	Speed	Cost	Result
1	**Refetch on tab focus** plus a slow timer	On focus: immediate	None. Existing endpoints.	**Chosen** (P4N-10).
2	Cheap change check (since)	Same as 1	One small endpoint (BL-01).	Backlog.
3	Server-Sent Events	Under 1 second	Open connection per tab, token refresh, gateway timeouts.	Backlog (BL-05).
4	WebSocket	Under 1 second	As option 3, and more complex.	Rejected.
5	FCM web push	Instant, tab closed	Service worker, VAPID key, permission, CSP.	Rejected. Chrome requires a visible notification.

Browsers slow timers in hidden tabs. For this reason the focus trigger matters more than the timer.

5. Testing locally

Target	Works locally?	How	Needs
Android emulator	Yes	Use a virtual device with a **Google Play** image. The app gets a real FCM token. Send a data message with curl or from your local backend.	Internet. Free Firebase project. google-services.json.
Android phone (USB)	Yes	Same as the emulator. Use it for background and battery tests.	Same.
Web	Not needed	Web does not use FCM. Test the focus refetch in any browser.	Nothing.
iOS simulator	Yes, later	`xcrun simctl push booted BUNDLE_ID payload.apns` for a fake push. A real remote push needs the Push capability and the APNs sandbox.	Apple Developer account for the real push.
iPhone (real)	Yes, but paid	Build, sign and run on the device.	Apple Developer Program, 99 USD per year.

Send a silent test message without the backend

```
# Get an OAuth token for the dev service account, then send one data message
curl -X POST \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  https://fcm.googleapis.com/v1/projects/PROJECT_ID/messages:send \
  -d '{"message":{"token":"DEVICE_TOKEN",
      "android":{"priority":"NORMAL","collapse_key":"sync"},
      "data":{"type":"SYNC","dictionaryId":"UUID"}}}'
```

- The Firebase console button "Send test message" always sends a **visible** notification. Use curl for silent tests.
- This Mac has no gcloud and no Firebase CLI. A small script can create the OAuth token from the service-account key.
- Watch `onMessageReceived` in Logcat to see the message arrive.

> **Do this first.** Create two Firebase projects: **verborum-dev** and **verborum-prod**. Keep separate config files and service accounts. A test message must never reach a real user.

6. Cost

Item	Cost	Needed for	Note
FCM messages	0 USD	Android, iOS	Listed as no-cost on the Firebase pricing page. Quota 600,000 messages per minute per project.
Firebase project	0 USD	Android, iOS	The free Spark plan is enough. Two projects (dev, prod) cost nothing.
Apple Developer Program	99 USD per year	iOS push, TestFlight, App Store	The same membership you need to publish. Push is a capability inside it. No second fee.
Backend compute	Small	ms_notification	One Spring Boot container and one small Postgres database. Cost depends on the hosting plan.
Web	0 USD	Web	No Firebase on web. Refetch uses the existing API.

Prices were read from vendor pages on 2026-10-13. Check them again before you decide.

7. Backend design (ms_notification)

Mirror ms_marketplace. Ship SecurityConfig in the first commit. The owner of a device record always comes from the JWT subject.

Data model (Liquibase JSON changesets, UUID string ids)

Table	Columns	Rules
`device_tokens`	device_id (PK, client UUID), fk_user_id (JWT subject), platform (ANDROID, IOS), token, locale, app_version, last_seen_at, creation_dt, update_dt	UNIQUE (token). Index on fk_user_id. No FK to another service. Upsert on PUT.
`notification_preferences`	Deferred (P4N-05).	Silent pushes have no user-visible category to switch off.

API

Method and path	Body	Result
`PUT /notifications/devices`	{deviceId, token, platform, locale, appVersion}	201. Idempotent upsert. A token that moves account is reassigned.
`DELETE /notifications/devices/{deviceId}`	none	200. No-op if absent or not yours. Call it on logout.

Event to message mapping

Event	Recipient	data.type	Status
`dictionary.changed`	The user's other devices	SYNC	First
`dictionary.imported` (+publisherId)	The publisher	REFRESH	Second
`dictionary.rated`	The publisher	REFRESH	Second

- **Skip the originator.** The event carries the optional deviceId of the writing device. That device gets nothing.
- **Coalesce bursts.** Collapse key `sync` per user, plus a short server-side debounce. One bulk request is one event.
- **No fan-out.** SYNC goes to the few devices of one user, so batching is not a problem.
- **Forum rule.** REFRESH goes only to a Forum member, and never to the actor.

Delivery rules (the seven messaging rules apply)

- Publish events **after commit**. One dead-lettered queue per routing key. `INFERRED` type precedence.
- Listeners are idempotent. A redelivery can send a second push. The collapse key makes it harmless.
- On FCM 429 or 5xx, throw. Retry and the dead-letter queue then work. On `UNREGISTERED`, delete the token.
- `user.deleted` removes every token of the user. Match on **keycloakId**.
- A job removes tokens not seen for 30 days. Clients register again on each launch and on token refresh.

Secrets

The service-account JSON is a secret. Read its path from `FIREBASE_CREDENTIALS_FILE` with `FIREBASE_PROJECT_ID`. Put placeholders in .env.example. Never commit the key. Never put it in an image or the Keycloak realm file. Without a key, the sender logs a warning and skips.

8. Client work

Android (verborum_android)

- Add the Google services plugin, the Firebase BoM and `firebase-messaging` to libs.versions.toml. Put google-services.json in app/.
- Create `VerborumMessagingService : FirebaseMessagingService` with Hilt. Declare it in the manifest.
- `onNewToken` enqueues a WorkManager job (network constraint) that sends PUT /notifications/devices.
- Register through a **PostLoginHook** and again on each launch. This updates last_seen_at.
- On logout: call DELETE, then `FirebaseMessaging.deleteToken()`.
- `onMessageReceived`: SYNC enqueues the existing sync worker. REFRESH invalidates the Forum cache. Do not sync inline.
- Ignore messages while the user is logged out.
- **No permission prompt. No notification channel.** Add both only when a visible alert is planned.

Web (verborum_kmp)

- Refetch the data of the open screen when the tab becomes visible (`visibilitychange`).
- Add a slow timer (about 60 seconds) that runs only while the tab is visible.
- Use existing endpoints. No service worker, token, VAPID key, permission or CSP change.
- Any web write already emits `dictionary.changed`, so Android hears about web edits.

iOS (verborum_kmp, later)

- Join the Apple Developer Program. Add the Push Notifications capability and Background Modes (Remote notifications) in Xcode.
- Upload an APNs auth key (.p8) to both Firebase projects. Add the Firebase iOS SDK with Swift Package Manager.
- Keep Swift thin. Define a `PushTokenRegistrar` interface in commonMain and bind it in Koin.
- Add no Android code. The project has no Android target.

9. Security and privacy

- **Ownership.** The device record belongs to the JWT subject. A client cannot register a token for another user.
- **Logout.** DELETE removes the token. A shared phone then stops syncing the previous user's data.
- **Payload.** Ids only. No dictionary names, word text or e-mail addresses.
- **Untrusted ids.** The app treats a dictionaryId from a push like any input. The API still returns 404 for an unreadable dictionary.
- **Least privilege.** In production, use a service account with only the Firebase Cloud Messaging API Admin role.
- **Privacy text.** Name Google (and later Apple) as processors of the device token.
- **Audit.** Run the security-auditor agent on ms_notification before you expose it.

10. Rollout plan (roadmap Phase 4N)

Stage	Tasks	Done when
1. Firebase and backend core	P4N-01 Firebase projects and secrets (owner). P4N-02 scaffold ms_notification. P4N-03 device tokens and endpoints. P4N-04 FCM sender. (P4N-05 preferences: deferred.)	A curl-registered token receives a data message.
2. Events and delivery	P4N-06 events (dictionary.changed, dictionary.rated, publisherId). P4N-07 consumers. P4N-08 docs and audit.	An edit on device A makes device B sync. A receives nothing.
3. Clients, local	P4N-09 Android. P4N-10 web refetch on focus (no dependency, build at any time).	Android syncs in the background. A web tab refreshes on focus.
4. Android and web release	P4N-11 release prerequisites (also SEC-03 and SEC-11).	A release build receives a push from the production stack.
5. iOS (needs Apple account)	P4N-12 iOS build. P4N-13 iOS release.	A TestFlight build receives a production push.

Next. Next step: you create the verborum-dev Firebase project, register the Android app (package de.coldtea.verborum.app) and save the service-account key outside the repos. Then P4N-02 starts.

11. Verification checklist

- A device registers once. A second PUT creates no new row.
- Logout removes the token. The phone then receives nothing for that user.
- An UNREGISTERED reply removes the token.
- An edit on device A makes device B sync in the background. Device A receives nothing.
- Ten quick edits deliver at most a few messages.
- With the app force-stopped or the push lost, opening the dictionary list still syncs everything.
- `user.deleted` removes every token of the user.
- A redelivered event does no harm.
- A web tab shows an Android edit on focus, or within one timer interval while visible.

12. Sources

Read on 2026-10-13. Vendor prices and limits can change.

- Firebase pricing (Cloud Messaging listed as no-cost): firebase.google.com/pricing
- FCM throttling and quotas (600,000 messages per minute): firebase.google.com/docs/cloud-messaging/throttling-and-quotas
- FCM token management (30-day and 270-day rules): firebase.google.com/docs/cloud-messaging/manage-tokens
- FCM error codes (4096-byte limit, UNREGISTERED, TTL): firebase.google.com/docs/cloud-messaging/error-codes
- FCM Android client: firebase.google.com/docs/cloud-messaging/android/client
- FCM send with Admin SDK: firebase.google.com/docs/cloud-messaging/send/admin-sdk
- Firebase Local Emulator Suite (no Messaging): firebase.google.com/docs/emulator-suite
- Apple Developer Program (99 USD, Push capability): developer.apple.com/programs/whats-included
- Remote push on the iOS simulator (Xcode 14): nilcoalescing.com/blog/TestingRemotePushOniOSSimulator
- OneSignal free plan change (1,000 MAU): pushwoosh.com/blog/onesignal-free-plan-changes

Not verified: the 500-token multicast limit, that xcrun simctl push works without a paid account, and Chrome's exact rule for a push with no visible notification. Check them in P4N-04, P4N-12 and, if web push returns, P4N-10.